

[Home](#)[GitHub](#)

CLASSES

[ArticlesService](#)[ErrorBoundary](#)

EVENTS

[SIGINT](#)[unhandledRejection](#)

NAMESPACES

[apiConfig](#)[contactoEmergenciaService](#)[diarioService](#)

GLOBAL

[404_Error_Handler](#)[ACTIVIDADES](#)[API_URL](#)[ASPECTOS_COGNITIVOS](#)[ActionCard](#)[ActivitySelector](#)[App](#)[Articulos](#)[AuthButtons](#)[AuthLayout](#)[Button](#)[CORS_Configuration](#)[Card](#)[Carousel](#)[Checkbox](#)[CircularProgress](#)[CognitionSelector](#)[Diario](#)[DiaryBanner](#)[DiaryEditor](#)[DiaryEntry](#)[EMOCIONES](#)[EmergencyButton](#)[EmergencyModal](#)[EmotionSelector](#)[EmotionStats](#)

backend/src/app.js

```
1.  /**
2.   * @file Fichero principal de la aplicación Express.
3.   * @description Configura y arranca el servidor Express, define midw
4.   * @requires http-errors
5.   * @requires express
6.   * @requires path
7.   * @requires cookie-parser
8.   * @requires morgan
9.   * @requires cors
10.  * @requires helmet
11.  * @requires observability.service - Servicio de monitoreo y logging
12.  */
13.
14.  var createError = require('http-errors');
15.  var express = require('express');
16.  var path = require('path');
17.  var cookieParser = require('cookie-parser');
18.  var logger = require('morgan');
19.  var cors = require('cors');
20.  var helmet = require('helmet');
21.  const { metricsMiddleware, getMetrics } = require('./services/observ
22.
23.  var app = express();
24.
25.  // view engine setup
26.  app.set('views', path.join(__dirname, 'views'));
27.  app.set('view engine', 'pug');
28.
29.  app.use(logger('dev'));
30.  app.use(helmet());
31.  app.use(metricsMiddleware);
32.
33.  /**
34.   * @name CORS_Configuration
35.   * @description Configuración de CORS para permitir peticiones desde
36.   * @property {string} origin - El origen permitido para las peticion
37.   * @property {boolean} credentials - Indica si se permiten credencia
38.   * @property {number} optionsSuccessStatus - Código de estado para p
39.   */
40.  // Configuración de CORS
41.  const corsOptions = {
42.    origin: process.env.FRONTEND_URL || 'http://localhost:3000', //
43.    credentials: true, // Permitir envío de cookies y headers de aut
44.    optionsSuccessStatus: 200
45.  };
46.  app.use(cors(corsOptions));
47.
48.  app.use(express.json());
49.  app.use(express.urlencoded({ extended: false }));
50.  app.use(cookieParser());
51.
52.  const authRouter = require('./routes/auth.routes');
53.  const formularioRouter = require('./routes/formulario.routes');
54.  const registroRouter = require('./routes/registro.routes');
```

```

55. const diarioRouter = require('./routes/diario.routes');
56. const iaRoutes = require('./routes/ia.routes');
57. const contactoEmergenciaRouter = require('./routes/contactoEmergencia.routes');
58. const healthRouter = require('./routes/health.routes');
59.
60. app.use('/api/auth', authRouter);
61. app.use('/api/formulario', formularioRouter);
62. app.use('/api/registro', registroRouter);
63. app.use('/api/registros', registroRouter); // Añadido para aceptar 1
64. app.use('/api/diario', diarioRouter);
65. app.use('/api/health', healthRouter);
66. app.use('/api', iaRoutes);
67. app.use('/api/contactos-emergencia', contactoEmergenciaRouter);
68.
69. /**
70.  * @name GET_/api/metrics
71.  * @description Endpoint para obtener métricas en formato Prometheus
72.  * @param {object} req - Objeto de petición de Express.
73.  * @param {object} res - Objeto de respuesta de Express.
74.  */
75. app.get('/api/metrics', async (req, res) => {
76.   res.set('Content-Type', 'text/plain; charset=utf-8');
77.   res.send(await getMetrics());
78. });
79.
80. //app.use('/', indexRouter);
81. //app.use('/users', usersRouter);
82.
83. /**
84.  * @name GET_/
85.  * @description Ruta de prueba para comprobar que el servidor funciona
86.  * @param {object} req - Objeto de petición de Express.
87.  * @param {object} res - Objeto de respuesta de Express.
88.  */
89. app.get('/', (req, res) => {
90.   res.send('Funciona')
91. })
92.
93.
94. /**
95.  * @name 404_Error_Handler
96.  * @description Middleware para capturar errores 404 (Not Found).
97.  * @param {object} req - Objeto de petición de Express.
98.  * @param {object} res - Objeto de respuesta de Express.
99.  * @param {function} next - Función para pasar al siguiente middleware
100.  */
101. // catch 404 and forward to error handler
102. app.use(function(req, res, next) {
103.   next(createError(404));
104. });
105.
106. /**
107.  * @name Generic_Error_Handler
108.  * @description Middleware para gestionar todos los demás errores.
109.  * @param {object} err - Objeto de error.
110.  * @param {object} req - Objeto de petición de Express.
111.  * @param {object} res - Objeto de respuesta de Express.
112.  * @param {function} next - Función para pasar al siguiente middleware
113.  */
114. // error handler
115. app.use(function(err, req, res, next) {
116.   // set locals, only providing error in development
117.   res.locals.message = err.message;
118.   res.locals.error = req.app.get('env') === 'development' ? err : {}

```

```
119.  
120.    // render the error page  
121.    res.status(err.status || 500);  
122.    res.render('error');  
123.  });  
124.  
125.  module.exports = app;
```

Documentation generated by [JSDoc 4.0.5](#) on Fri Feb 13 2026 09:08:14 GMT+0000 (Coordinated Universal Time) using the [docdash](#) theme.